BÁO CÁO ASSIGNMENT 01

Thông tin	Chi tiết
Sinh viên	Nguyễn Ngọc Quang
MSSV	SE202007V
Ngày thực hiện	10/10/2026

MỤC LỤC

1. Tổng quan và môi trường thực hiện....3
1.1. Cấu trúc mã nguồn....3
2. Sơ đồ kiến trúc (Mục 1)....4
2.1. Kiến trúc tổng thể hai driver....4
2.2. Luồng xử lý ioctl....4
3. So sánh ba loại device driver (Mục 2)....6
3.1. Quyết định thiết kế cụ thể....6
4. Phân tích thiết kế hai driver (Mục 3)....7
4.1. register_chrdev() và misc_register()....7
4.2. Tại sao dùng atomic_t thay vì int....7
4.3. Cơ chế ioctl magic number và mã hóa lệnh....8
5. Kết quả kiểm thử (Mục 4)....10
5.1. Task 1 – test_asgn1.sh....10
5.2. Task 2 – test_counter.sh....10
5.3. Kernel log (dmesg)....11
5.4. Lỗi gặp phải và cách debug....12
6. Câu hỏi lý thuyết (Task 4)....14
Câu 1. Reference counting của module....14
Câu 2. Tại sao GET_STATS phải dùng copy_to_user()....14
Câu 3. MTD khác gì block device....15
Câu 4. Xử lý *ppos trong counter_read()....15
7. Kết luận....17

(Nếu mục lục trống: trong LibreOffice/Word chuột phải vào mục lục → Update Index / Update Field.)

1. Tổng quan và môi trường thực hiện

Assignment gồm hai driver độc lập chạy trên QEMU vexpress-a9: **asgn1_driver** mở rộng character driver của LAB-02 với giao diện ioctl gồm 4 lệnh, và **counter_driver** là một misc device mô phỏng bộ đếm phần cứng, có `/proc/counter_info`. Cả hai được kiểm thử tự động bằng shell script trong QEMU, kết quả **9/9 test đạt** cho mỗi driver.

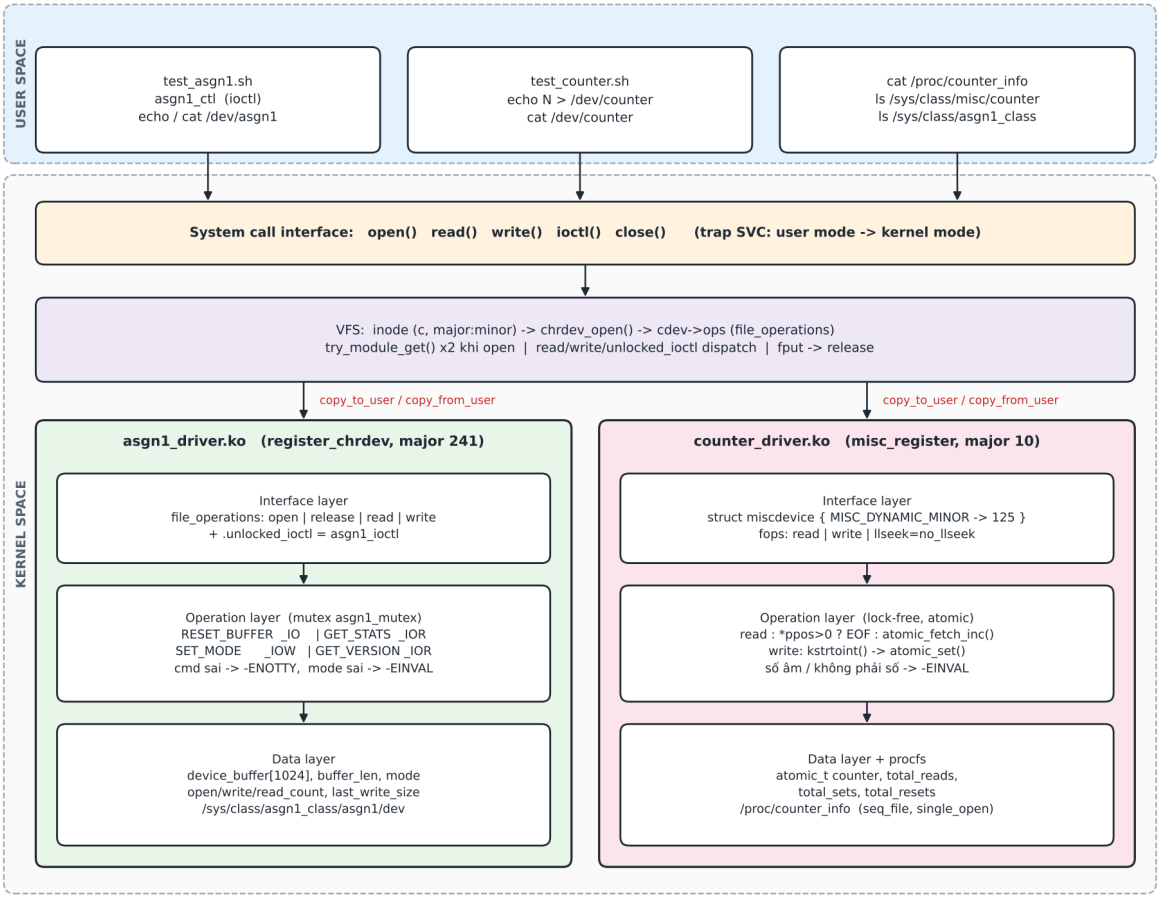
Thành phần	Phiên bản / cấu hình
Host	Ubuntu (x86_64), GCC 15.2.0, GNU Binutils 2.46
Compiler cho kernel + module	arm-linux-gnueabihf-gcc-12 (Ubuntu 12.5.0)
Compiler cho userspace (asgn1_ctl)	arm-linux-gnueabihf-gcc 15.2.0, link tĩnh (-static)
Kernel target	Linux 5.15.0, vexpress_defconfig (CONFIG_MODULES, MODULE_UNLOAD, DEVTMPFS = y)
vermagic module	5.15.0 SMP mod_unload ARMv7 p2v8
Root filesystem	initramfs BusyBox (kế thừa LAB-02), init qua /etc/inittab + rcS
Major number	asgn1: 241 (cố định) | counter: 10 (MISC_MAJOR), minor 125 (cấp động)

1.1. Cấu trúc mã nguồn

File	Vai trò
task1_ioctl/asgn1_ioctl.h	Header dùng chung kernel + userspace: magic 'q', struct asgn1_stats, 4 macro ioctl
task1_ioctl/asgn1_driver.c	Character driver (register_chrdev), read/write có mutex, unlocked_ioctl
task1_ioctl/asgn1_ctl.c	Công cụ userspace gọi ioctl (shell không gọi trực tiếp được ioctl)
task1_ioctl/test_asgn1.sh	9 test tự động: mode, 4 ioctl, ENOTTY/EINVAL, reset, rmmod
task2_misc/counter_driver.c	Misc device, atomic_t counter, /proc/counter_info (seq_file)
task2_misc/test_counter.sh	9 test: auto-increment, set/reset, EINVAL, /proc tại 3 thời điểm

2. Sơ đồ kiến trúc (Mục 1)

2.1. Kiến trúc tổng thể hai driver



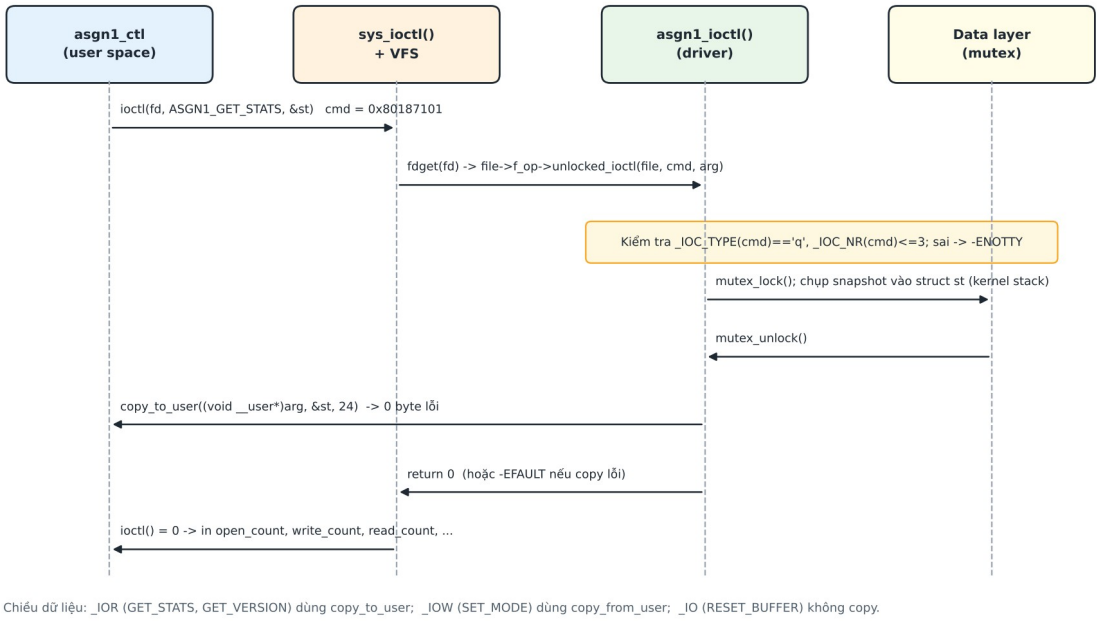
Hình 1. Luồng userspace → system call → VFS → file_operations → dữ liệu driver, procfs/sysfs

Mỗi driver được tách thành ba lớp như yêu cầu của đề:

- **Interface layer:** phần đăng ký với kernel. asgn1 dùng `register_chrdev(241)` + `class_create` + `device_create` để có `/sys/class/asgn1_class/asgn1` và node `/dev/asgn1`. counter dùng `struct miscdevice` + `misc_register()`, kernel tự tạo `/sys/class/misc/counter` và `/dev/counter`.
- **Operation layer:** các hàm trong `file_operations`. Mọi dữ liệu đi qua ranh giới user/kernel đều dùng `copy_to_user()` / `copy_from_user()`.
- **Data layer:** trạng thái của driver. asgn1 bảo vệ bằng một `mutex`; counter dùng `atomic_t` nên không cần khóa.

VFS là điểm trung gian: khi `open("/dev/asgn1")`, inode mang số major:minor 241:0, `chrdev_open()` tìm cdev tương ứng và gán `file->f_op = &asgn1_fops`. Từ đó mọi `read/write/ioctl` trên file descriptor được VFS chuyển thẳng tới hàm của driver.

2.2. Luồng xử lý ioctl



Hình 2. Luồng ASGN1_GET_STATS từ userspace tới driver và trở về

Điểm thiết kế quan trọng trong `GET_STATS`: driver **chụp snapshot** các counter vào biến `st` trên stack kernel khi đang giữ mutex, mở khóa, rồi mới `copy_to_user()`. Như vậy mutex không bị giữ trong lúc copy (copy có thể gây page fault và ngủ), và userspace nhận được một bộ số liệu nhất quán tại cùng một thời điểm.

Trích asgn1_driver.c – xử lý GET_STATS

```
case ASGN1_GET_STATS:
      mutex_lock(&asgn1_mutex);
      st.open_count      = open_count;
      st.write_count     = write_count;
      st.read_count      = read_count;
      st.buffer_len      = buffer_len;
      st.last_write_size = last_write_size;
      st.mode            = mode;
      mutex_unlock(&asgn1_mutex);

      if (copy_to_user(uarg, &st, sizeof(st)))
            return -EFAULT;
      pr_info("asgn1: ioctl GET_STATS\n");
      return 0;
```

3. So sánh ba loại device driver (Mục 2)

Tiêu chí	Character device	Block device	Network device
Giao diện với kernel	struct file_operations (open, read, write, unlocked_ioctl, mmap…)	struct block_device_operations + blk-mq (queue_rq xử lý struct request/bio)	struct net_device_ops (ndo_open, ndo_start_xmit, ndo_stop…)
Có node /dev?	Có: /dev/asgn1, /dev/counter	Có: /dev/mmcblk0, /dev/sda	Không; truy cập qua socket, tên interface eth0, can0
Đơn vị dữ liệu	Luồng byte (byte stream), tuần tự	Block/sector cố định (512 B, 4 KB), truy cập ngẫu nhiên	Packet/frame (sk_buff)
Buffering	Không có cache chung; driver tự quản lý buffer (vd device_buffer[1024])	Có page cache + I/O scheduler (mq-deadline, kyber), gom và sắp xếp request	Hàng đợi TX/RX (qdisc), ring buffer DMA, NAPI
Ai gọi driver	Process qua syscall trực tiếp	Filesystem (ext4, squashfs) qua block layer	Network stack (TCP/IP, SocketCAN)
Ví dụ trong hệ nhúng	UART, GPIO, I2C/SPI sensor, watchdog, ADC	eMMC, SD card, USB mass storage	Ethernet (smsc911x trên vexpress), CAN, Wi-Fi

3.1. Quyết định thiết kế cụ thể

Ví dụ 1 – Cảm biến nhiệt độ I2C trên ECU: dữ liệu là vài byte đọc theo yêu cầu, không có khái niệm vị trí hay khối, không cần filesystem. Chọn **character device** (cụ thể là misc device như counter_driver), dùng `read()` để lấy giá trị và `ioctl()` để cấu hình chu kỳ lấy mẫu. Dùng block device ở đây là vô nghĩa vì không có sector để lưu trữ.

Ví dụ 2 – Bộ điều khiển CAN trong xe: dữ liệu là các frame có ID, cần hàng đợi, timestamp và nhiều ứng dụng cùng nhận. Chọn **network device** theo chuẩn SocketCAN: ứng dụng dùng `socket(PF_CAN)`, có sẵn filter, qdisc và công cụ `candump`. Nếu viết thành char device thì phải tự làm lại toàn bộ phần hàng đợi và phân phối frame.

Ví dụ 3 – Bộ nhớ eMMC chứa rootfs: filesystem ext4 cần đọc/ghi ngẫu nhiên theo khối và hưởng lợi từ page cache. Chọn **block device**. (Ngược lại, raw NAND không có FTL phải đi qua MTD – xem câu 3 phần 6.)

4. Phân tích thiết kế hai driver (Mục 3)

4.1. register_chrdev() và misc_register()

Tiêu chí	register_chrdev() – Task 1	misc_register() – Task 2
Major number	Tự chọn (241) hoặc 0 để cấp động; có thể trùng với driver khác	Luôn là 10 (MISC_MAJOR), không tốn major mới
Minor number	Chiếm cả 256 minor của major	Một minor; MISC_DYNAMIC_MINOR → kernel cấp (thực tế: 125)
Tạo /sys và /dev	Phải tự gọi class_create() + device_create()	Tự động tạo /sys/class/misc/counter và /dev/counter
Số bước init/exit	3 bước, cần goto cleanup ngược thứ tự khi lỗi	1 lệnh gọi misc_register / misc_deregister
Phù hợp khi	Driver có nhiều minor (nhiều kênh), cần class riêng, hoặc cần kiểm soát major	Driver nhỏ, một node, đơn mục đích: watchdog, RNG, bộ đếm, cảm biến đơn

Với Task 1, đề yêu cầu major cố định trong khoảng 241–254 nên `register_chrdev()` là bắt buộc. Đoạn init minh họa chi phí của cách này: mỗi bước có thể lỗi và phải dọn dẹp các bước trước đó.

Trích asgn1_init() – cleanup theo thứ tự ngược

```
ret = register_chrdev(DRIVER_MAJOR, DEVICE_NAME, &asgn1_fops);
if (ret < 0)
      return ret;
asgn1_class = class_create(THIS_MODULE, CLASS_NAME);
if (IS_ERR(asgn1_class)) { ret = PTR_ERR(asgn1_class); goto err_chrdev; }
asgn1_device = device_create(asgn1_class, NULL, MKDEV(DRIVER_MAJOR, 0), NULL,
DEVICE_NAME);
if (IS_ERR(asgn1_device)) { ret = PTR_ERR(asgn1_device); goto err_class; }
return 0;
err_class:
      class_destroy(asgn1_class);
err_chrdev:
      unregister_chrdev(DRIVER_MAJOR, DEVICE_NAME);
      return ret;
```

Với counter_driver, toàn bộ phần đăng ký chỉ là một struct và một lệnh gọi. Log thực tế xác nhận kernel cấp minor 125: `counter: registered /dev/counter major=10 minor=125`.

Trích counter_driver.c

```
static struct miscdevice counter_misc = {
      .minor = MISC_DYNAMIC_MINOR,
      .name  = DEV_NAME,          /* -> /dev/counter */
      .fops  = &counter_fops,
      .mode  = 0666,
};
ret = misc_register(&counter_misc);
```

4.2. Tại sao dùng atomic_t thay vì int

Câu lệnh `counter++` với `int` được biên dịch thành 3 lệnh ARM tách rời: load từ bộ nhớ vào thanh ghi, cộng, store ngược lại. QEMU chạy với 2 CPU (`SMP: Total of 2 processors activated`), nên hai process gọi `read()` cùng lúc có thể xen kẽ như sau:

Thời điểm	CPU0 (process A: cat)	CPU1 (process B: cat)	counter trong RAM
t1	LDR r0, [counter] → r0 = 5		5
t2		LDR r0, [counter] → r0 = 5	5
t3	ADD r0, #1; STR → 6		6
t4		ADD r0, #1; STR → 6	6 (mất 1 lần tăng)
Kết quả	A nhận "5"	B cũng nhận "5"	Hai lần đọc trả cùng số, counter chỉ tăng 1

`atomic_t` khắc phục bằng các lệnh độc quyền `LDREX/STREX` của ARMv7: STREX chỉ thành công nếu không CPU nào ghi vào địa chỉ đó kể từ LDREX, nếu thất bại thì lặp lại. Ngoài ra, driver dùng `atomic_fetch_inc()` (lấy giá trị cũ **và** tăng trong một thao tác) thay vì `atomic_read()` rồi `atomic_inc()`: nếu tách hai lệnh, mỗi lệnh tuy nguyên tử nhưng cặp lệnh thì không, vẫn có thể có hai reader nhận cùng một số. Với một biến đơn, atomic nhẹ hơn mutex vì không ngủ và không có chi phí khóa.

Trích counter_read()

```
if (*ppos > 0)
      return 0;                          /* EOF cho lần read thứ 2 */
val = atomic_fetch_inc(&counter);     /* đọc + tăng nguyên tử*/
n = snprintf(kbuf, sizeof(kbuf), "%d\n", val);
if (copy_to_user(buf, kbuf, n))
      return -EFAULT;
*ppos += n;
```

4.3. Cơ chế ioctl magic number và mã hóa lệnh

Mỗi mã ioctl là một số 32 bit được ghép từ 4 trường bằng macro `_IOC(dir, type, nr, size)`:

Bit	31–30	29–16	15–8	7–0
Trường	dir (hướng)	size (kích thước tham số)	type (magic)	nr (số thứ tự)
Ý nghĩa	0 = none, 1 = write (_IOW), 2 = read (_IOR), 3 = cả hai	sizeof(kiểu dữ liệu), tối đa 16 KB	Ký tự nhận diện driver	Phân biệt lệnh trong driver

Công thức: `cmd = (dir << 30) | (size << 16) | (type << 8) | nr`. Các macro giải mã ngược: `_IOC_NR(cmd) = cmd & 0xFF`, `_IOC_TYPE(cmd) = (cmd >> 8) & 0xFF`, `_IOC_SIZE(cmd) = (cmd >> 16) & 0x3FFF`, `_IOC_DIR(cmd) = cmd >> 30`. Kết quả thực tế in bởi `asgn1_ctl info` trên QEMU:

Lệnh (asgn1_ioctl.h)	Mã hex	dir	size	type	nr
_IO(ASGN1_MAGIC, 0) – RESET_BUFFER	0x00007100	0	0	'q' = 0x71	0
_IOR(…, 1, struct asgn1_stats) – GET_STATS	0x80187101	2	24	0x71	1
_IOW(…, 2, int32_t) – SET_MODE	0x40047102	1	4	0x71	2
_IOR(…, 3, char[64]) – GET_VERSION	0x80407103	2	64	0x71	3

Ví dụ tách `GET_STATS = 0x80187101`: hai bit cao `10` → dir = 2 (read); bit 29–16 = `0x0018` = 24 byte, đúng bằng `sizeof(struct asgn1_stats)` gồm 6 trường `int32_t`; byte `0x71` là ký tự 'q'; byte thấp `0x01` là lệnh số 1.

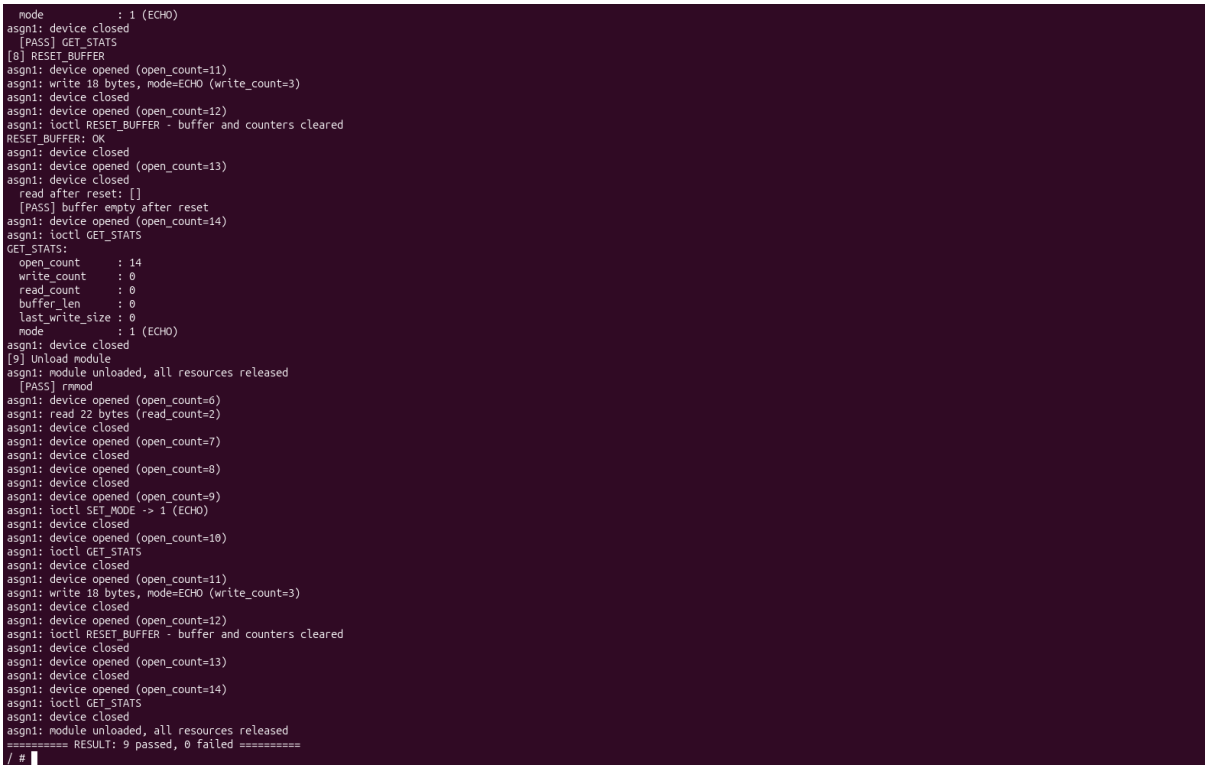
Tại sao cần magic number: mọi driver đều nhận ioctl qua cùng một syscall. Nếu hai driver cùng dùng số lệnh 1, một chương trình mở nhầm device sẽ khiến driver kia thực hiện một thao tác ngoài ý muốn. Magic number khiến mã lệnh gần như duy nhất trên toàn hệ thống; driver kiểm tra `_IOC_TYPE(cmd) != ASGN1_MAGIC` và trả `-ENOTTY` ("inappropriate ioctl for device"). Trường `size` còn giúp phát hiện userspace và kernel dùng hai phiên bản struct khác nhau. Driver đưa lên mainline phải đăng ký magic trong `Documentation/userspace-api/ioctl/ioctl-number.rst`; với driver nội bộ của bài, 'q' được chọn và kiểm chứng bằng test `[6]`: lệnh `_IO('q', 99)` bị từ chối với `ENOTTY`.

5. Kết quả kiểm thử (Mục 4)

5.1. Task 1 – test_asgn1.sh

#	Nội dung test	Kết quả quan sát	Đạt
1	insmod, tạo /dev/asgn1	lsmod: asgn1_driver 16384 0; /dev/asgn1 = c 241,0	PASS
3	GET_VERSION (_IOR, 64 byte)	asgn1_driver v1.0 - SE202007V	PASS
4	Echo mode (mặc định)	read: [Hello Echo]	PASS
5	SET_MODE 0 → Kernel mode	read: [[KERNEL] Hello Kernel]	PASS
6	SET_MODE 5; ioctl _IO('q',99)	EINVAL; ENOTTY	PASS ×2
7	GET_STATS	open 10, write 2, read 2, buf_len 22, last 13, mode 1	PASS
8	Ghi → RESET_BUFFER → đọc	read after reset: [] ; counters về 0	PASS
9	rmmod	asgn1: module unloaded, all resources released	PASS

Tổng: 9 passed, 0 failed. Các số liệu GET_STATS khớp với log: 2 lần ghi ("Hello Echo" 11 byte, "Hello Kernel" 13 byte → `last_write_size = 13`), `buffer_len = 22` = 9 byte tiền tố `"[KERNEL] "` + 13 byte dữ liệu.

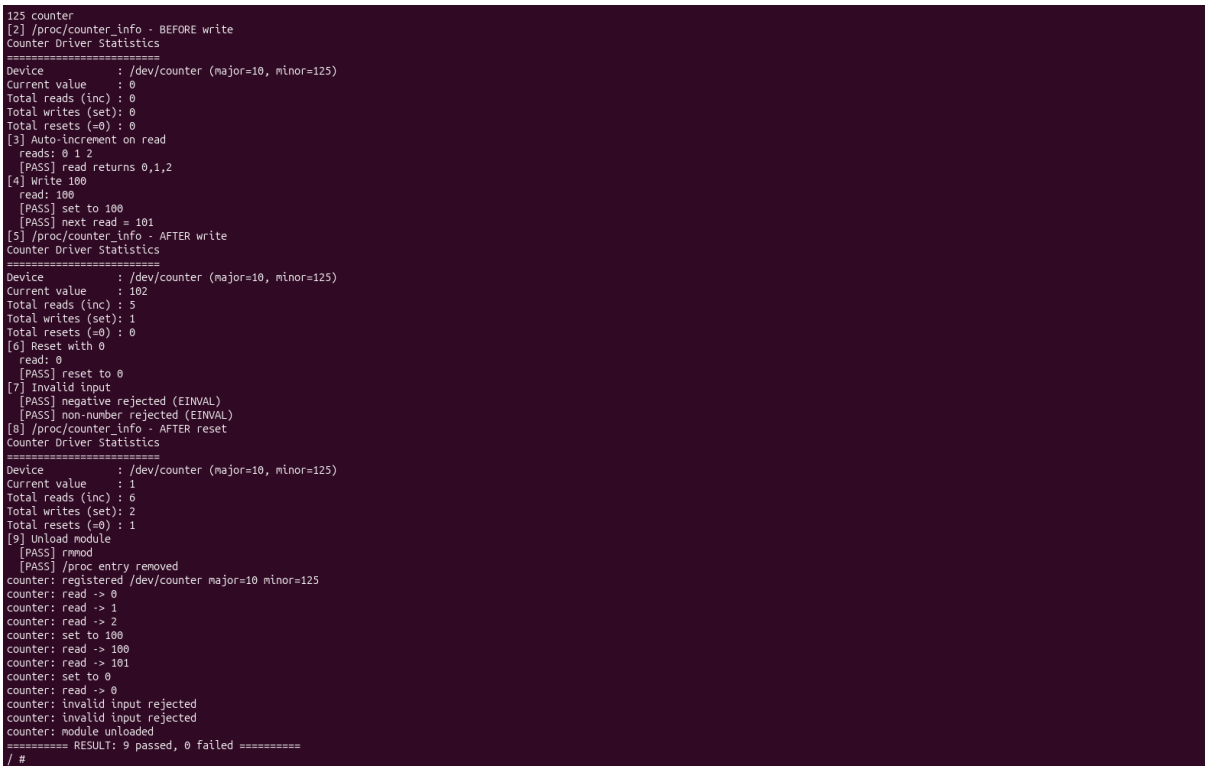


Hình 3. test_asgn1.sh chạy trên QEMU – RESULT: 9 passed, 0 failed

5.2. Task 2 – test_counter.sh

Ba lần đọc liên tiếp trả `0 1 2`; ghi `100` thì đọc ra 100 rồi 101; ghi `0` thì đọc ra 0; ghi `-5` và `abc` đều bị từ chối với `EINVAL`. **Tổng: 9 passed, 0 failed**. Nội dung `/proc/counter_info` tại ba thời điểm:

Trường	Trước write	Sau write 100	Sau reset (0)
Current value	0	102	1
Total reads (inc)	0	5	6
Total writes (set)	0	1	2
Total resets (=0)	0	0	1



Hình 4. test_counter.sh chạy trên QEMU – /proc/counter_info tại 3 thời điểm, RESULT: 9 passed, 0 failed

Giá trị 102 và 1 là đúng với đặc tả *auto-increment on read*: sau khi ghi 100, test đọc hai lần (nhận 100 và 101) nên counter hiện tại là 102; sau khi reset về 0, test đọc một lần (nhận 0) nên counter là 1. Hai lần ghi không hợp lệ không làm tăng `Total writes`, chứng tỏ driver từ chối trước khi thay đổi trạng thái.

5.3. Kernel log (dmesg)



Hình 5. dmesg trên QEMU – vòng đời đầy đủ của asgn1_driver và counter_driver

Mỗi ioctl và mỗi thao tác đọc/ghi đều có một dòng log tương ứng, và mỗi `insmod` đều có dòng `unloaded` khớp, xác nhận tài nguyên được giải phóng. Dòng *taints kernel* là bình thường với module out-of-tree.

5.4. Lỗi gặp phải và cách debug

Lỗi 1: Không build được kernel – "selected processor does not support `dmb ish'"

Triệu chứng: sau khi build lại kernel từ đầu, hàng trăm lỗi assembler kiểu `selected processor does not support 'dmb ish' / 'ldrex' / 'wfi' in ARM mode`. Đây đều là lệnh ARMv7, nghĩa là assembler đang tưởng CPU là đời cũ hơn. Đổi sang `gcc-12` vẫn lỗi, nên nguyên nhân không nằm ở bản GCC.

Debug: build một file với `V=1` để xem cờ thực sự truyền cho compiler:

```
make: *** [Makefile:1868: arch/arm/plat-versatile] Error 2
/tmp/ccUaxDsW.s: Assembler messages:
/tmp/ccUaxDsW.s:1299: Error: selected processor does not support `dmb ish' in ARM mode
/tmp/ccUaxDsW.s:1321: Error: architectural extension `mp' is not allowed for the current base architecture
/tmp/ccUaxDsW.s:1322: Error: selected processor does not support `pldw [r3]' in ARM mode
/tmp/ccUaxDsW.s:1325: Error: selected processor does not support `pld [r3]' in ARM mode
/tmp/ccUaxDsW.s:1337: Error: selected processor does not support `ldrex r2,[r3]' in ARM mode
/tmp/ccUaxDsW.s:1339: Error: selected processor does not support `strex r1,r2,[r3]' in ARM mode
/tmp/ccUaxDsW.s:1353: Error: selected processor does not support `dmb ish' in ARM mode
make[1]: *** [scripts/Makefile.build:277: init/do_mounts.o] Error 1
make[1]: *** Waiting for unfinished jobs....
/tmp/ccIHmhjt.s: Assembler messages:
/tmp/ccIHmhjt.s:4316: Error: selected processor does not support `cpsid i' in ARM mode
/tmp/ccIHmhjt.s:5150: Error: selected processor does not support `cpsid i' in ARM mode
/tmp/ccIHmhjt.s:5434: Error: selected processor does not support `cpsie i' in ARM mode
/tmp/ccIHmhjt.s:5601: Error: selected processor does not support `dsb' in ARM mode
/tmp/ccIHmhjt.s:6095: Error: selected processor does not support `cpsie i' in ARM mode
/tmp/ccIHmhjt.s:6690: Error: selected processor does not support `cpsid i' in ARM mode
/tmp/ccIHmhjt.s:6737: Error: selected processor does not support `dmb ishst' in ARM mode
/tmp/ccIHmhjt.s:6783: Error: selected processor does not support `dmb ishst' in ARM mode
/tmp/ccIHmhjt.s:6867: Error: selected processor does not support `dmb ish' in ARM mode
/tmp/ccIHmhjt.s:6888: Error: selected processor does not support `dsb ishst' in ARM mode
/tmp/ccIHmhjt.s:6892: Error: selected processor does not support `sev' in ARM mode
/tmp/ccIHmhjt.s:6986: Error: architectural extension `mp' is not allowed for the current base architecture
/tmp/ccIHmhjt.s:6987: Error: selected processor does not support `pldw [r4]' in ARM mode
/tmp/ccIHmhjt.s:6990: Error: selected processor does not support `pld [r4]' in ARM mode
/tmp/ccIHmhjt.s:7004: Error: selected processor does not support `ldrex r3,[r4]' in ARM mode
/tmp/ccIHmhjt.s:7006: Error: selected processor does not support `strex r1,r2,[r4]' in ARM mode
make[1]: *** [scripts/Makefile.build:277: init/main.o] Error 1
make: *** [Makefile:1868: init] Error 2
quang@quang-VivoBook-ASUSLaptop-X513EAN-A515EA:~/embedded_lab1/kernel/linux-5.15$
```

Hình 6. Triệu chứng: build kernel bằng GCC 15 báo lỗi selected processor does not support

```
quang@quang-VivoBook-ASUSLaptop-X513EAN-A515EA:~/embedded_lab1/kernel/linux-5.15$ cd ~/embedded_lab1/kernel/linux-5.15
grep -n "CONFIG_CPU_32v7)" arch/arm/Makefile
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- CC=arm-linux-gnueabihf-gcc-12 init/main.o V=1 2>&1 \
  | grep -o -- '-march=[^ ]*' | sort -u
64:arch-$(CONFIG_CPU_32v7)              =-D__LINUX_ARM_ARCH__=7 $(call cc-option,-march=armv7-a,-march=armv5t -Wa$(comma)-march=armv7-a)
-march=armv5t
-march=armv7-a
quang@quang-VivoBook-ASUSLaptop-X513EAN-A515EA:~/embedded_lab1/kernel/linux-5.15$
```

Hình 7. Debug bằng V=1: kernel dùng -march=armv5t, dòng 64 của arch/arm/Makefile

Nguyên nhân: dòng 64 của `arch/arm/Makefile` là `$(call cc-option,-march=armv7-a,-march=armv5t -Wa$(comma)-march=armv7-a)`. Phép thử `cc-option` thất bại với toolchain hard-float của Ubuntu, nên kernel lùi về `-march=armv5t`. GCC ghi `.arch armv5t` vào file assembly, và binutils 2.46 tôn trọng chỉ thị này hơn tham số `-Wa`, nên từ chối lệnh ARMv7.

Sửa: ép kernel dùng ARMv7 trực tiếp, và build kernel lẫn module bằng cùng compiler `gcc-12`:

```
sed -i 's|^arch-$(CONFIG_CPU_32v7).*|arch-$(CONFIG_CPU_32v7)\t=-D__LINUX_ARM_ARCH__=7
-march=armv7-a|' \
    arch/arm/Makefile
# Makefile của module:  KCC := $(CROSS_COMPILE)gcc-12 ;  ... CC=$(KCC) modules
```

Sau khi sửa, kernel build thành công (`Linux version 5.15.0 ... arm-linux-gnueabihf-gcc-12 ... GNU ld 2.46`) và module có vermagic khớp.



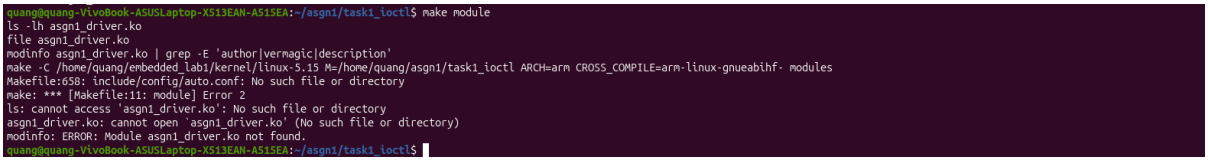
Hình 8. Sau khi sửa: Kernel: arch/arm/boot/zImage is ready



Hình 9. Module build bằng gcc-12: ARM EABI5, vermagic 5.15.0 SMP mod_unload ARMv7

Lỗi 2: include/config/auto.conf: No such file or directory

Lần `make module` đầu tiên thất bại vì thư mục kernel source của LAB-01 không còn `.config` (đã bị clean). Module ngoài cây cần kernel đã được cấu hình và build để có header sinh tự động và `Module.symvers`. Cách sửa: `make vexpress_defconfig` rồi build lại `zImage modules dtbs`. Hệ quả phụ: module `lab2_driver.ko` của LAB-02 không còn khớp kernel mới, nên rcS của assignment được chỉnh để bỏ qua nó.



Hình 10. make module lỗi: include/config/auto.conf: No such file or directory



Hình 11. Chẩn đoán: kernel source không còn .config

Lỗi 3 (thiết kế): read() của LAB-02 không giữ mutex

Khi chuyển code từ LAB-02, phát hiện `lab2_read()` đọc `buffer_len` và `device_buffer` mà không khóa, trong khi `lab2_write()` có khóa. Trên hệ 2 CPU, một process có thể đọc đúng lúc process khác đang `copy_from_user` vào buffer, nhận về dữ liệu nửa cũ nửa mới, hoặc đọc `buffer_len` mới với nội dung cũ. Trong asgn1_driver, `asgn1_read()` giữ mutex suốt quá trình và dùng một điểm thoát `goto out` duy nhất để không quên mở khóa.

6. Câu hỏi lý thuyết (Task 4)

Câu 1. Reference counting của module

Mỗi `struct module` có một bộ đếm tham chiếu. `try_module_get()` tăng và `module_put()` giảm. Bộ đếm **tăng** khi: một file của driver được mở (VFS lấy tham chiếu tới `fops->owner`), một module khác dùng symbol do module này export, hoặc code kernel chủ động giữ module. Bộ đếm **giảm** khi file được đóng lần cuối (`fput → release → module_put`) hoặc module phụ thuộc được gỡ. `delete_module()` (rmmod) chỉ thành công khi bộ đếm bằng 0.

Thực nghiệm trên QEMU:



Hình 12. Thực nghiệm refcount trên QEMU: Used by 0 → 2 → 0, rmmod bị từ chối khi fd còn mở

Quan sát đáng chú ý là chỉ một lần mở mà refcount tăng **2**. Nguyên nhân nằm ở `chrdev_open()`: `cdev_get()` gọi `try_module_get(cdev->owner)` (với `register_chrdev()`, kernel gán `cdev->owner = fops->owner = THIS_MODULE`), rồi `fops_get()` lại gọi `try_module_get(fops->owner)` lần nữa. Cả hai tham chiếu được trả lại khi fd đóng, nên số đếm về 0.

Khi còn process giữ file, rmmod báo `Resource temporarily unavailable` (EAGAIN/EWOULDBLOCK): BusyBox rmmod gọi `delete_module()` với cờ `O_NONBLOCK`, kernel thấy refcount ≠ 0 nên từ chối ngay thay vì chờ. Cách gỡ đúng là tìm và dừng process đang giữ file (`fuser /dev/asgn1`, `lsof`) rồi rmmod lại. Cách ép buộc là `rmmod -f`, nhưng chỉ dùng được khi kernel bật `CONFIG_MODULE_FORCE_UNLOAD` (vexpress_defconfig không bật) và rất nguy hiểm: process vẫn giữ con trỏ tới `file_operations` nằm trong vùng code đã bị giải phóng, lần gọi `read()` tiếp theo sẽ gây kernel oops.

Câu 2. Tại sao GET_STATS phải dùng copy_to_user()

Tham số `arg` của ioctl là một **địa chỉ ảo của process**, chỉ có nghĩa trong bảng trang của process đó. Trên ARM 32-bit với cấu hình mặc định, 3 GB thấp là vùng user, 1 GB cao là vùng kernel. `memcpy()` hay dereference trực tiếp giả định địa chỉ đích luôn hợp lệ, điều này sai vì nhiều lý do:

1. **Địa chỉ có thể chưa được map** (trang chưa cấp phát, bị swap, hoặc copy-on-write). Khi kernel ghi vào, MMU sinh data abort. `copy_to_user()` có bảng *exception table*: nếu lỗi xảy ra ở lệnh truy cập user, handler sẽ thử xử lý page fault bình thường, và nếu địa chỉ thật sự không hợp lệ thì nhảy tới đoạn fixup và trả về số byte chưa copy (driver trả `-EFAULT`). `memcpy()` không có mục trong exception table, nên fault trong kernel mode trở thành **kernel oops**.

2. **Bảo mật:** userspace có thể truyền một địa chỉ thuộc vùng kernel. `copy_to_user()` gọi `access_ok()` để kiểm tra địa chỉ nằm dưới `TASK_SIZE`; `memcpy()` sẽ ghi đè bộ nhớ kernel theo ý kẻ tấn công.
3. **Ở cấp MMU (ARMv7):** khi bật `CONFIG_CPU_SW_DOMAIN_PAN`, kernel đặt domain user thành *no access* trong lúc chạy code kernel bình thường, chỉ mở trong các hàm uaccess. Một lệnh ghi thẳng vào địa chỉ user sẽ gây **domain fault** ngay cả khi trang đó hợp lệ, cơ chế này tương tự PAN trên ARMv8 và chính là để bắt lỗi kiểu dùng memcpy.

Ngoài ra, struct `st` được điền trên stack kernel rồi copy một lần 24 byte, nên userspace không bao giờ thấy trạng thái dở dang và kernel không giữ mutex trong lúc có thể page fault.

Câu 3. MTD khác gì block device

Tiêu chí	Block device	MTD (Memory Technology Device)
Mô hình truy cập	Đọc/ghi sector bất kỳ, ghi đè trực tiếp	Ba thao tác: read, write (chỉ đổi bit 1 → 0), erase cả erase block
Ai lo wear-leveling, bad block	Bộ điều khiển trong thiết bị (FTL của eMMC/SD/SSD)	Phần mềm: UBI/UBIFS, JFFS2 hoặc driver
Filesystem phù hợp	ext4, FAT, squashfs	JFFS2, UBIFS (hoặc squashfs trên mtdblock read-only)
Node	/dev/mmcblk0, /dev/sda	/dev/mtd0 (char), /dev/mtdblock0 (giả lập block)

Một thiết bị phải đi qua MTD khi nó là **flash thô không có bộ điều khiển FTL**: raw NAND gắn vào NAND controller của SoC, NOR flash song song, và SPI-NOR chứa bootloader/firmware trên các board nhúng. Với loại này, hệ điều hành phải tự erase trước khi ghi, tự phân bổ đều số lần erase và tự đánh dấu bad block. Đặt ext4 trực tiếp lên đó sẽ ghi đè cùng một block liên tục (ví dụ vùng journal) và làm hỏng flash nhanh chóng.

Bằng chứng ngay trong boot log của QEMU: board vexpress có hai chip NOR flash được driver `physmap-flash` đăng ký qua MTD và ghép thành một thiết bị (`Concatenating MTD devices: (0) "40000000.flash" (1) "40000000.flash"`), không phải block device. Ngược lại, khe SD (`mmci-pl18x`) xuất hiện như block device `mmcblk` vì thẻ SD có FTL bên trong.

Câu 4. Xử lý *ppos trong counter_read()

`cat` không biết trước file dài bao nhiêu: nó gọi `read(fd, buf, 4096)` lặp lại, in những gì nhận được, và **chỉ dừng khi read() trả về 0** (EOF) hoặc trả lỗi. Ở phía kernel, `vfs_read()` truyền con trỏ tới vị trí hiện tại `file->f_pos` vào hàm read của driver dưới dạng `loff_t *ppos`, và sau khi driver trả về thì lưu giá trị mới vào `f_pos`. Vì vậy driver dùng `*ppos` để biết đây là lần đọc đầu hay đã đọc rồi.

- Driver đúng: lần 1 `*ppos = 0` → trả `"0\n"` (2 byte), `*ppos = 2`. Lần 2 `*ppos = 2 > 0` → trả 0, `cat` gặp EOF và thoát. Mỗi lần chạy `cat` chỉ in đúng một số.

- Bỏ kiểm tra `*ppos`: lần nào driver cũng trả một số mới và tăng counter, `read()` không bao giờ trả 0, nên `cat` lặp vô hạn, in 0, 1, 2, 3… và counter tăng mãi. Lệnh chỉ dừng khi bị Ctrl+C (SIGINT) hoặc khi đầu đọc của pipe đóng (SIGPIPE).

Thực nghiệm: build bản thử nghiệm `counter_noppos.ko` thay `if (*ppos > 0)` bằng `if (0)`:

```
quang@quang-VivoBook-ASUSLaptop-X513EAN-A515EA:~/asgn1/logs$ mkdir -p ~/asgn1/q4_test && cd ~/asgn1/q4_test
sed -e 's/^\tif (\*ppos > 0)$/\tif (0) \/* Q4: ppos check disabled *\//' \
    -e 's/"counter_info"/"counter_noppos_info"/' \
    ~/asgn1/task2_misc/counter_driver.c > counter_noppos.c
grep -n "Q4: ppos" counter_noppos.c
cp ~/asgn1/task2_misc/Makefile . && sed -i 's/counter_driver.o/counter_noppos.o/' Makefile
make && file counter_noppos.ko
49:     if (0) /* Q4: ppos check disabled */
make -C /home/quang/embedded_lab1/kernel/linux-5.15 M=/home/quang/asgn1/q4_test ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- CC=arm-linux-gnueabihf-gcc-12 modules
counter_noppos.ko: ELF 32-bit LSB relocatable, ARM, EABI5 version 1 (SYSV), BuildID[sha1]=9db03dc71672d85cfe2b3d1b04c032c6ad2a464c, with debug_info, not stripped
quang@quang-VivoBook-ASUSLaptop-X513EAN-A515EA:~/asgn1/q4_test$
```

*Hình 13. Build bản thử nghiệm counter_noppos.ko (if (*ppos > 0) → if (0))*

```
/ # dmesg -n 1
/ # echo "--- WITH ppos check (correct driver) ---"
--- WITH ppos check (correct driver) ---
/ # insmod /lib/modules/5.15.0/counter_driver.ko
/ # cat /dev/counter
0
/ # cat /dev/counter
1
/ # rmmod counter_driver
/ #
/ # echo "--- WITHOUT ppos check (Q4 test) ---"
--- WITHOUT ppos check (Q4 test) ---
/ # insmod /lib/modules/5.15.0/counter_noppos.ko
/ # cat /dev/counter | head -n 15
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
/ # rmmod counter_noppos
```

*Hình 14. Driver đúng: mỗi cat in 1 số; bản bỏ kiểm tra *ppos: một lần cat in liên tục 0 → 14*

Với driver đúng, mỗi `cat` in một số rồi dừng. Với bản lỗi, **một lần `cat` duy nhất** in liên tục 0 → 14 và chỉ dừng vì `head -n 15` đóng pipe làm `cat` nhận SIGPIPE. Driver cũng đặt `.llseek = no_llseek` vì vị trí trong file không có ý nghĩa với một bộ đếm.

7. Kết luận

Hai driver đáp ứng đầy đủ yêu cầu chức năng và kỹ thuật của đề: asgn1_driver có header ioctl riêng, xử lý đủ 4 lệnh với `-ENOTTY` cho lệnh sai và mutex bảo vệ toàn bộ dữ liệu; counter_driver dùng `misc_register`, `atomic_t` và xử lý `*ppos` đúng. Cả hai đạt 9/9 test trên QEMU ARMv7, không rò rỉ tài nguyên sau rmmod.

Bài học lớn nhất đến từ phần không nằm trong đề: một toolchain mới hơn kernel có thể làm hỏng build theo cách khó đoán, và cách hiệu quả để tìm nguyên nhân là xem cờ build thật (`V=1`) thay vì đổi công cụ một cách mò mẫm. Thực nghiệm refcount cũng cho thấy giá trị của việc quan sát thực tế: lý thuyết thường nói "tăng 1 khi open", còn kernel 5.15 thực ra tăng 2 qua `cdev_get` và `fops_get`.

[1]